2026-10-20 MySQL表的约束：上

- 1. MySQL 表的约束（上）
  - 1.1 为什么需要表的约束
  - 1.2 空属性：NULL 与 NOT NULL
    - 1.2.1 NULL 的含义
    - 1.2.2 NOT NULL 的作用
  - 1.3 默认值：DEFAULT
    - 1.3.1 DEFAULT 的生效条件
    - 1.3.2 NOT NULL 与 DEFAULT 的关系
  - 1.4 列描述：COMMENT
  - 1.5 补零显示：ZEROFILL
    - 1.5.1 整数括号中的长度不是存储范围
    - 1.5.2 ZEROFILL 的显示效果
    - 1.5.3 使用注意
  - 1.6 主键约束：PRIMARY KEY
    - 1.6.1 主键的核心性质
    - 1.6.2 建表时直接指定主键
    - 1.6.3 主键冲突
    - 1.6.4 后添加与删除主键
    - 1.6.5 复合主键
    - 1.6.6 主键设计检查
  - 1.7 上篇小结与下篇衔接
    - 1.7.1 五类知识点对照
    - 1.7.2 本篇实验最容易混淆的地方
    - 1.7.3 从“上篇”走到“下篇”

1. MySQL 表的约束（上）

1.1 为什么需要表的约束

数据类型能够限制字段“能存哪一类数据”，但这种限制通常比较单一。例如，`INT` 可以限制取值范围，却无法说明一个邮箱是否与别人重复，也无法说明学生必须属于一个真实存在的班级。因此，还需要表级和列级的额外规则，从业务逻辑角度保证数据的合法性与正确性。

约束的作用可以概括为：

- 让未来插入、更新到表中的数据符合预期。
- 通过数据库的强制检查倒逼程序员写入正确的数据。
- 保证数据的完整性和可预期性。
- 让表结构本身表达业务规则，而不完全依赖应用代码检查。

一份好的表结构不只是“留下一些可以存放数据的格子”，还要在数据库入口处主动拒绝不符合规则的数据。

本课程涉及的常见约束如下：

约束或属性	主要解决的问题	本篇位置
`NULL` / `NOT NULL`	字段是否允许缺失	本篇
`DEFAULT`	省略字段时使用什么值	本篇
`COMMENT`	如何为字段补充说明	本篇
`ZEROFILL`	整数如何补零显示	本篇
`PRIMARY KEY`	如何唯一且非空地标识一行	本篇
`AUTO_INCREMENT`	如何自动生成整数编号	下篇
`UNIQUE KEY`	如何保证业务值不重复	下篇
`FOREIGN KEY`	如何保证跨表引用有效	下篇

1.2 空属性：`NULL` 与 `NOT NULL`

1.2.1 `NULL` 的含义

创建字段时，如果不写是否允许为空，数据库通常允许该字段取 `NULL`。`NULL` 表示“没有值”“值未知”或“尚未赋值”，不能简单等同于数字 `0` 或空字符串 `''`。

`NULL` 的特殊性还体现在运算中：

```
SELECT NULL;
SELECT 1 + NULL;
```

两个结果都是 `NULL`。未知值参与普通算术运算后，结果仍然是未知值。因此，如果某列在业务上必须参与计算，通常应尽量声明为 `NOT NULL`，并在设计数据时给它一个明确值。

判断空值时应使用 `IS NULL` 或 `IS NOT NULL`：

```
SELECT * FROM myclass WHERE class_name IS NULL;
SELECT * FROM myclass WHERE class_name IS NOT NULL;
```

不能使用 `class_name = NULL` 来判断空值。

1.2.2 `NOT NULL` 的作用

`NOT NULL` 要求该字段必须有明确值，插入或更新时不能把它设为 `NULL`。

课堂案例要求设计一张班级表，同时保存班级名和上课教室：

- 如果班级没有名字，就不知道是哪一个班。
- 如果教室没有名字，就不知道在哪里上课。
- 这些缺失值都不符合正常业务逻辑，因此两列都应当声明为 `NOT NULL`。

```
CREATE TABLE myclass (
    class_name VARCHAR(20) NOT NULL,
    class_room VARCHAR(10) NOT NULL
);
```

查看表结构：

```
DESC myclass;
```

```
+------------+-------------+------+-----+---------+-------+
| Field      | Type        | Null | Key | Default | Extra |
+------------+-------------+------+-----+---------+-------+
| class_name | varchar(20) | NO   |     | NULL    |       |
| class_room | varchar(10) | NO   |     | NULL    |       |
+------------+-------------+------+-----+---------+-------+
```

`Null` 列的 `NO` 表示不允许空值。虽然 `Default` 显示为 `NULL`，但这里并不存在一个可用的默认值；省略字段赋值时仍会报错。

```
mysql> create table if not exists myclass(
    -> class_name varchar(20) not null,
    -> class_root varchar(20) not null,
    -> other varchar(20)
    -> );
Query OK, 0 rows affected (0.02 sec)

mysql> desc myclass;
+------------+-------------+------+-----+---------+-------+
| Field      | Type        | Null | Key | Default | Extra |
+------------+-------------+------+-----+---------+-------+
| class_name | varchar(20) | NO   |     | NULL    |       |
| class_root | varchar(20) | NO   |     | NULL    |       |
| other      | varchar(20) | YES  |     | NULL    |       |
+------------+-------------+------+-----+---------+-------+
3 rows in set (0.00 sec)

mysql> show create table myclass\G
*************************** 1. row ***************************
       Table: myclass
Create Table: CREATE TABLE `myclass` (
  `class_name` varchar(20) NOT NULL,
  `class_root` varchar(20) NOT NULL,
  `other` varchar(20) DEFAULT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8
1 row in set (0.00 sec)
```

如果只给班级名，省略了没有默认值的教室：

```
INSERT INTO myclass(class_name) VALUES('class1');
```

MySQL 会拒绝插入：

```
ERROR 1364 (HY000): Field 'class_room' doesn't have a default value
```

正确写法需要同时提供两个非空字段：

```
INSERT INTO myclass(class_name, class_room)
VALUES('class1', 'room101');
```

需要注意，`NOT NULL` 只保证“必须有值”，不能保证“值在业务上正确”。例如，`class_room VARCHAR(10) NOT NULL` 可以阻止空教室，但仍无法判断 `'不存在的教室'` 是否为真实教室；后一种约束需要额外规则或外键来解决。

1.3 默认值：`DEFAULT`

1.3.1 `DEFAULT` 的生效条件

如果某一种数据经常使用同一个具体值，可以在建表时通过 `DEFAULT` 指定默认值。插入数据时省略该字段，MySQL 才会使用默认值。

课堂示例：

```
CREATE TABLE tt10 (
    name VARCHAR(20) NOT NULL,
    age TINYINT UNSIGNED DEFAULT 0,
    sex CHAR(2) DEFAULT '男'
);
```

查看表结构时可看到 `age` 与 `sex` 的默认值：

```
DESC tt10;
```

```
+-------+--------------------+------+-----+---------+-------+
| Field | Type               | Null | Key | Default | Extra |
+-------+--------------------+------+-----+---------+-------+
| name  | varchar(20)        | NO   |     | NULL    |       |
| age   | tinyint(3) unsigned | YES  |     | 0       |       |
| sex   | char(2)            | YES  |     | 男      |       |
+-------+--------------------+------+-----+---------+-------+
```

只插入 `name`：

```
INSERT INTO tt10(name) VALUES('zhangsan');
SELECT * FROM tt10;
```

结果中，省略的字段自动采用默认值：

```
+----------+------+------+
| name     | age  | sex  |
+----------+------+------+
```

```
| zhangsan |    0 | 男    |
+----------+------+------+
```

```
mysql> create table if not exists t11( name varchar(20) not null, age tinyint unsigned default 18, gender char(1) default '男' );
Query OK, 0 rows affected (0.01 sec)

mysql> desc t11;
+--------+---------------------+------+-----+---------+-------+
| Field  | Type                | Null | Key | Default | Extra |
+--------+---------------------+------+-----+---------+-------+
| name   | varchar(20)         | NO   |     | NULL    |       |
| age    | tinyint(3) unsigned | YES  |     | 18      |       |
| gender | char(1)             | YES  |     | 男      |       |
+--------+---------------------+------+-----+---------+-------+
3 rows in set (0.01 sec)
```

这里的关键是：`DEFAULT` 在“该列被省略”时生效。若插入时明确写入了其他合法值，则使用用户提供的值；若显式写入 `NULL`，则要结合列是否允许空来判断，不能把“显式 `NULL`”和“完全省略该列”混为一谈。

> 只有设置了默认值的列，省略该列时才可能成功；没有默认值且允许空值的列，省略后通常得到 `NULL`；既不允许空值又没有默认值的列，省略后会报错。

1.3.2 `NOT NULL` 与 `DEFAULT` 的关系

`NOT NULL` 与 `DEFAULT` 并不冲突，它们约束的是两件不同的事：

属性	约束重点
`NOT NULL`	不允许最终写入 `NULL`
`DEFAULT`	省略该字段时提供一个预置值

二者可以配合使用：

```sql
CREATE TABLE account (
    id INT NOT NULL,
    status TINYINT NOT NULL DEFAULT 1
);
```

这样可以同时表达“状态字段不可缺失”和“新建记录默认启用”。

> 提到“`NOT NULL` 和 `DEFAULT` 一般不需要同时出现，因为 `DEFAULT` 本身有默认值，不会为空”。这句话只说明设置默认值后，通常不需要再额外提供值；从约束设计角度看，两者仍然是在分别处理“省略字段”和“禁止 `NULL`”，实际项目中可以且经常需要同时使用。

区别可以用下面的例子理解：

```
CREATE TABLE default_demo (
    a INT DEFAULT 10,
    b INT NOT NULL
);
```

- `a` 没有 `NOT NULL`，省略时得到 `10`，显式写 `NULL` 仍可能成功。
- `b` 有 `NOT NULL` 但没有默认值，省略时会直接报错。
- 若业务既要求必须填写、又希望省略时自动补值，应写成 `INT NOT NULL DEFAULT 10`。

```
mysql> desc t12;
+--------+---------------------+------+-----+---------+-------+
| Field  | Type                | Null | Key | Default | Extra |
+--------+---------------------+------+-----+---------+-------+
| name   | varchar(20)         | NO   |     | NULL    |       |
| age    | tinyint(3) unsigned | YES  |     | 18      |       |
| gender | char(1)             | NO   |     | 男      |       |
+--------+---------------------+------+-----+---------+-------+
3 rows in set (0.00 sec)

mysql> insert into t12(name ,age,gender) values ('张三',20,null);
ERROR 1048 (23000): Column 'gender' cannot be null
mysql> insert into t12(name ,age,gender) values ('张三',20);
ERROR 1136 (21S01): Column count doesn't match value count at row 1
mysql> insert into t12(name ,age,gender) values ('张三',20,'男');
Query OK, 1 row affected (0.00 sec)

mysql> insert into t12(name ,age) values ('张三',20);
Query OK, 1 row affected (0.00 sec)
```

```
mysql> create table t13(name varchar(20),age tinyint(3));
Query OK, 0 rows affected (0.01 sec)

mysql> show create table t13\G;
*************************** 1. row ***************************
       Table: t13
Create Table: CREATE TABLE `t13` (
  `name` varchar(20) DEFAULT NULL,
  `age` tinyint(3) DEFAULT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8
1 row in set (0.00 sec)
```

1.4 列描述：`COMMENT`

`COMMENT` 用于为字段补充说明，例如字段业务含义、单位和填写规则。它没有数据校验作用，也不会改变字段类型，但会随建表语句保存，方便程序员或 DBA 理解表结构。

```
CREATE TABLE tt12 (
    name VARCHAR(20) NOT NULL COMMENT '姓名',
    age TINYINT UNSIGNED DEFAULT 0 COMMENT '年龄',
    sex CHAR(2) DEFAULT '男' COMMENT '性别'
);
```

`COMMENT` 与代码注释类似，属于便于维护的元数据，不是约束。插入一个不符合注释描述的值时，数据库不会仅凭注释自动拒绝。

使用 `DESC` 只能看到字段名、类型、空值、键、默认值和扩展信息，看不到注释：

```
DESC tt12;
```

要查看注释，需要查看完整建表语句：

```
SHOW CREATE TABLE tt12\G
```

输出中会保留类似下面的字段定义：

```
`name` varchar(20) NOT NULL COMMENT '姓名',
`age` tinyint(3) unsigned DEFAULT '0' COMMENT '年龄',
`sex` char(2) DEFAULT '男' COMMENT '性别'
```

```
mysql> desc t14;
+--------+---------------------+------+-----+---------+-------+
| Field  | Type                | Null | Key | Default | Extra |
+--------+---------------------+------+-----+---------+-------+
| name   | varchar(20)         | NO   |     | NULL    |       |
| age    | tinyint(3) unsigned | YES  |     | 18      |       |
| gender | char(1)             | YES  |     | 男      |       |
+--------+---------------------+------+-----+---------+-------+
3 rows in set (0.00 sec)

mysql> select * from t14;
+--------+------+--------+
| name   | age  | gender |
+--------+------+--------+
| 张三   |   25 | 男     |
+--------+------+--------+
1 row in set (0.00 sec)

mysql> show create table t14\G;
*************************** 1. row ***************************
       Table: t14
Create Table: CREATE TABLE `t14` (
  `name` varchar(20) NOT NULL COMMENT '这个是用户名',
  `age` tinyint(3) unsigned DEFAULT '18' COMMENT '这个是用户的年龄',
  `gender` char(1) DEFAULT '男' COMMENT '这个是用户的性别'
) ENGINE=InnoDB DEFAULT CHARSET=utf8
1 row in set (0.00 sec)
```

因此，`COMMENT` 的价值在于让表结构自带说明，尤其是在字段较多、多人协作或长期维护的项目中。它不能替代数据类型、`NOT NULL`、默认值或后续的唯一键、外键等真正的约束。

1.5 补零显示：`ZEROFILL`

1.5.1 整数括号中的长度不是存储范围

早期 MySQL 可能在 `DESC` 或建表语句中把整数显示为 `INT(10)`、`TINYINT(3)` 等形式。括号里的数字主要是显示宽度相关信息，并不代表整数实际只能存 10 位或 3 位。

例如：

```sql
CREATE TABLE tt3 (
    a INT(10) UNSIGNED DEFAULT NULL,
    b INT(10) UNSIGNED DEFAULT NULL
);
```

```
INSERT INTO tt3 VALUES(1, 2);
SELECT * FROM tt3;
```

此时两列只显示原始数字：

```
+------+------+
| a    | b    |
+------+------+
|    1 |    2 |
+------+------+
```

没有 `ZEROFILL` 时，`INT(10)` 中的 `10` 并不会把所有数据都补齐到十位，也不会扩大 `INT` 的取值范围。

```
mysql> create table if not exists t15(
    -> a int unsigned not null,
    -> b int unsigned not null
    -> );
Query OK, 0 rows affected (0.01 sec)

mysql> desc t17;
ERROR 1146 (42S02): Table 'test2db.t17' doesn't exist
mysql> desc t15;
+-------+------------------+------+-----+---------+-------+
| Field | Type             | Null | Key | Default | Extra |
+-------+------------------+------+-----+---------+-------+
| a     | int(10) unsigned | NO   |     | NULL    |       |
| b     | int(10) unsigned | NO   |     | NULL    |       |
+-------+------------------+------+-----+---------+-------+
2 rows in set (0.00 sec)

mysql> show create table t15 \G
*************************** 1. row ***************************
       Table: t15
Create Table: CREATE TABLE `t15` (
  `a` int(10) unsigned NOT NULL,
  `b` int(10) unsigned NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8
1 row in set (0.00 sec)
```

1.5.2 `ZEROFILL` 的显示效果

把 `a` 改为宽度为 5 且带 `ZEROFILL` 的整数：

```
ALTER TABLE tt3
CHANGE a a INT(5) UNSIGNED ZEROFILL;

SELECT * FROM tt3;
```

查询结果变成：

```
+-------+------+
| a     | b    |
+-------+------+
| 00001 |    2 |
+-------+------+
```

原本的 `1` 被显示成 `00001`。规则是：

- 显示宽度不足时，在左侧补 `0`。
- 实际数值位数超过显示宽度时，不会因为显示宽度较小而截断真实数据。
- `ZEROFILL` 改变的是显示格式，数值本身没有变成 `00001`。

```
mysql> alter table t15 modify b int unsigned zerofill not null;
Query OK, 0 rows affected (0.05 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql> desc t15;
+-------+-----------------------------+------+-----+---------+-------+
| Field | Type                        | Null | Key | Default | Extra |
+-------+-----------------------------+------+-----+---------+-------+
| a     | int(10) unsigned            | NO   |     | NULL    |       |
| b     | int(10) unsigned zerofill | NO   |     | NULL    |       |
+-------+-----------------------------+------+-----+---------+-------+
2 rows in set (0.00 sec)

mysql> select * from t15;
+---+------------+
| a | b          |
+---+------------+
| 1 | 0000000002 |
+---+------------+
1 row in set (0.00 sec)
```

可以使用十六进制函数验证内部保存的仍然是数字 `1`：

```
SELECT a, HEX(a) FROM tt3;
```

```
+-------+--------+
| a     | hex(a) |
+-------+--------+
| 00001 | 1      |
+-------+--------+
```

`HEX(a)` 返回 `1`，说明数据库实际存储的值仍是数值 `1`，`00001` 只是格式化后的展示结果。

1.5.3 使用注意

- `ZEROFILL` 主要适合订单号、编号等希望等宽显示的场景。
- 它是显示属性，不应被当成业务编号生成机制。
- 在 MySQL 中，`ZEROFILL` 通常还会使整数按无符号方式处理；设计时要同时确认是否会丢失负数语义。
- 较新的 MySQL 版本中，整数显示宽度已经逐渐弱化；理解它的关键仍是“不改变实际存储值”。
- 真正需要固定格式的业务编号，应明确字段长度和生成规则，不能只依赖视觉补零。

1.6 主键约束：`PRIMARY KEY`

1.6.1 主键的核心性质

主键用于唯一标识表中的一条记录。它有四个关键性质：

- 主键值不能重复。
- 主键值不能为 `NULL`。
- 一张表最多只能有一个主键。
- 主键可以由一列组成，也可以由多列组成，但整体仍只算一个主键。

主键所在的列通常是整数类型，例如学号、商品编号、订单编号等。稳定且尽量与业务无关的主键，能让后续业务调整时少改表结构。

主键的意义不只在于“不能重复”，还在于只要拿到一个主键值，就能准确找到唯一一条对应的数据。若没有唯一标识，两条姓名相同、年龄相同的数据可能无法区分。

1.6.2 建表时直接指定主键

```
CREATE TABLE tt13 (
    id INT UNSIGNED PRIMARY KEY COMMENT '学号不能为空',
    name VARCHAR(20) NOT NULL
);
```

查看表结构：

```
DESC tt13;
```

```
+-------+------------------+------+-----+---------+-------+
| Field | Type             | Null | Key | Default | Extra |
+-------+------------------+------+-----+---------+-------+
| id    | int(10) unsigned | NO   | PRI | NULL    |       |
| name  | varchar(20)      | NO   |     | NULL    |       |
+-------+------------------+------+-----+---------+-------+
```

`id` 的 `Key` 为 `PRI`，`Null` 为 `NO`。这说明声明主键后，MySQL 会自动加上非空要求，不需要再单独写 `NOT NULL`。

```
mysql> create table if not exists test_key( id int unsigned primary key comment '学号', name varchar(20) not null );
Query OK, 0 rows affected (0.01 sec)

mysql> desc test_key;
+-------+-----------------+------+-----+---------+-------+
| Field | Type            | Null | Key | Default | Extra |
+-------+-----------------+------+-----+---------+-------+
| id    | int(10) unsigned | NO   | PRI | NULL    |       |
| name  | varchar(20)     | NO   |     | NULL    |       |
+-------+-----------------+------+-----+---------+-------+
2 rows in set (0.00 sec)

mysql>
```

```
mysql> show  create table test_key\G
*************************** 1. row ***************************
       Table: test_key
Create Table: CREATE TABLE `test_key` (
  `id` int(10) unsigned NOT NULL COMMENT '学号',
  `name` varchar(20) NOT NULL,
  PRIMARY KEY (`id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8
1 row in set (0.00 sec)

mysql>
```

1.6.3 主键冲突

正常插入一条数据：

```
INSERT INTO tt13 VALUES(1, 'aaa');
```

再次插入相同主键：

```
INSERT INTO tt13 VALUES(1, 'aaa');
```

MySQL 会拒绝：

```
ERROR 1062 (23000): Duplicate entry '1' for key 'PRIMARY'
```

这就是主键约束的实际效果。其他字段即使不同，只要主键相同，也不能再插入：

```
INSERT INTO tt13 VALUES(1, 'bbb');
```

同样会发生主键冲突。

1.6.4 后添加与删除主键

如果表创建时没有主键，之后可以在表上添加：

```
ALTER TABLE 表名 ADD PRIMARY KEY(字段列表);
```

例如：

```
ALTER TABLE test_key ADD PRIMARY KEY(id);
```

主键也可以删除：

```
ALTER TABLE test_key DROP PRIMARY KEY;
```

但不要等表运行了一段时间、已经积累重复数据后才添加主键。若目标列中本来就有重复值，添加主键会失败，往往还需要先清理数据。主键设计应当尽量在建表阶段确定。

删除主键只是删除这组主键约束，不等同于删除整个字段，也不等同于删除表中的数据。删除后，该列是否还能为 `NULL` 取决于字段自身的定义和其他约束。

1.6.5 复合主键

“一张表最多一个主键”并不表示“一个主键只能放在一列上”。一个主键可以包含多个字段，这种主键称为复合主键。

课堂选课表：

```
CREATE TABLE tt14 (
    id INT UNSIGNED,
    course CHAR(10) COMMENT '课程代码',
    score TINYINT UNSIGNED DEFAULT 60 COMMENT '成绩',
    PRIMARY KEY(id, course)
);
```

这里的主键是 `(id, course)` 这一组字段，而不是单独的 `id` 或单独的 `course`。

查看表结构：

```
DESC tt14;
```

```
+--------+---------------------+------+-----+---------+-------+
| Field  | Type                | Null | Key | Default | Extra |
+--------+---------------------+------+-----+---------+-------+
| id     | int(10) unsigned    | NO   | PRI | 0       |       |
| course | char(10)            | NO   | PRI |         |       |
| score  | tinyint(3) unsigned | YES  |     | 60      |       |
+--------+---------------------+------+-----+---------+-------+
```

两列的 `Key` 都是 `PRI`，表示它们共同组成一个复合主键。由于主键禁止 `NULL`，参与复合主键的两列也都自动成为非空字段。

```
mysql> desc pick_course;
+-----------+---------------------+------+-----+---------+-------+
| Field     | Type                | Null | Key | Default | Extra |
+-----------+---------------------+------+-----+---------+-------+
| id        | int(10) unsigned    | NO   | PRI | NULL    |       |
| course_id | int(10) unsigned    | NO   | PRI | NULL    |       |
| score     | tinyint(3) unsigned | YES  |     | NULL    |       |
+-----------+---------------------+------+-----+---------+-------+
3 rows in set (0.00 sec)
```

插入一条选课记录：

```
INSERT INTO tt14(id, course) VALUES(1, '123');
```

再次插入完全相同的组合：

```
INSERT INTO tt14(id, course) VALUES(1, '123');
```

会报主键冲突：

```
ERROR 1062 (23000): Duplicate entry '1-123' for key 'PRIMARY'
```

复合主键的规则是“整组值唯一”，而不是每个字段单独唯一：

- 同一个学生可以选多门课，因此 `id` 可以重复。
- 同一门课可以被多个学生选择，因此 `course` 也可以重复。
- 同一个学生不能重复选择同一门课，因此 `(id, course)` 不能重复。

这与“一个人在同一门课上的两条记录”在业务上没有意义是相呼应的。若业务还需要限制其他字段的组合不重复，可以在下篇继续学习 `UNIQUE KEY`。

1.6.6 主键设计检查

设计主键时可以依次检查：

1. 该字段是否能够在整个生命周期内唯一标识一条记录？
2. 该字段是否一定非空？
3. 业务变化时，该值是否可能被修改？
4. 它是单列主键还是复合主键？复合主键能否准确表达真实业务关系？
5. 是否把“只要求业务字段不重复”误当成主键？这种情况往往更适合下篇的唯一键。

1.7 上篇小结与下篇衔接

1.7.1 五类知识点对照

知识点	核心作用	关键结论
`NULL` / `NOT NULL`	控制字段能否缺失	`NULL` 是未知或无值，不参与普通运算；业务必备字段应优先声明 `NOT NULL`
`DEFAULT`	为省略字段提供默认值	与 `NOT NULL` 职责不同，可以配合使用
`COMMENT`	给字段补充维护说明	属于元数据，不提供强制校验
`ZEROFILL`	整数补零显示	只改变显示格式，不改变实际存储值
`PRIMARY KEY`	唯一、非空地标识记录	每表最多一个；可以是单列或复合主键；自动禁止 `NULL`

1.7.2 本篇实验最容易混淆的地方

- `DESC` 中看到 `Default = NULL`，不代表该列使用了默认值；还要结合 `Null = NO/YES` 理解。
- `NOT NULL` 和 `DEFAULT` 不是互相替代关系。
- `INT(5)` 不等于只能存五位数字。
- `00001` 只可能是补零后的显示结果，实际值仍可能是 `1`。
- 一张表只能有一个主键，但一个主键可以包含多列。
- 复合主键约束的是整组值不重复，不是每个组成字段都不能重复。
- 主键并不能自动保证业务字段不重复，例如手机号、邮箱等字段还需要下篇的唯一键。

1.7.3 从“上篇”走到“下篇”

上篇已经解决了“如何限制空值、默认值、字段说明和唯一行标识”这几类基础问题。但真实业务还有三个直接需求：

1. 主键值如何自动生成，避免每次手工找最大值？
2. 一张表只有一个主键，其他业务字段如何也保证不重复？
3. 一个学生的班级编号如何保证一定指向真实存在的班级？

它们分别对应下一篇的 `AUTO_INCREMENT`、`UNIQUE KEY` 和 `FOREIGN KEY`。尤其是自增长通常与主键配合使用，正好承接本篇的主键知识；唯一键则补充主键不能满足的多字段业务唯一性；外键再把表与表之间的关联检查交给数据库完成。

> 约束的本质不是增加麻烦，而是把业务规则提前写进表结构，让数据库在数据写入时主动守住正确性边界。